Flatten Binary Tree

AlgoExpert · Very Hard · Binary Trees

The problem

- Rewire the tree in place into a doubly linked list in in-order order (left = prev, right = next); return the leftmost node.

Intuition

- In-order linking: each subtree returns its leftmost and rightmost nodes so neighbours can be connected.

Approach

- 1. Flatten left and right recursively
- 2. Link left's rightmost to the node
- 3. Link the node to right's leftmost
- 4. Return the outer ends

Java solution

```java
public class FlattenBinaryTree {
    public static BinaryTree flattenBinaryTree(BinaryTree root) { return flatten(root)[0]; }

    /** returns {leftmost, rightmost} of the flattened subtree */
    private static BinaryTree[] flatten(BinaryTree n) {
        BinaryTree leftMost = n, rightMost = n;
        if (n.left != null) {
            BinaryTree[] l = flatten(n.left);
            l[1].right = n; n.left = l[1]; leftMost = l[0];
        }
        if (n.right != null) {
            BinaryTree[] r = flatten(n.right);
            r[0].left = n; n.right = r[0]; rightMost = r[1];
        }
        return new BinaryTree[]{leftMost, rightMost};
    }
}
```

Complexity

- Time: O(n)
- Space: O(h)

Recap

- In-order linking: each subtree returns its leftmost and rightmost nodes so neighbours can be connected.
- Complexity: O(n)